

Math Operators

+

-

*

/

//

= integer div

%

= remainder

** = power

$$10 // 3 = 3$$

$$10 \div 3 = 1$$

$$3 \overline{)10} \quad \text{3} = 10$$

9

1

Odd	3	2	= 1
	4	2	= 0
Even	5	2	= 1
	6	2	= 0

Math order

1. () 2. ** 3. *, /, //, % 4. +, -

ex.

$$16 ** 1 / 2$$

vs. $16 / 2 \rightarrow 8$

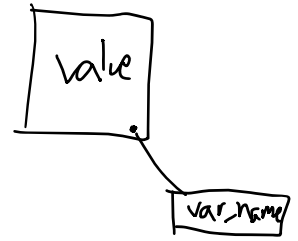
$$16 ** (1 / 2)$$

$16^{0.5} \rightarrow \sqrt{16} = 4$

Variables

↳ store and retrieve info

↳ Syntax: var_name = value



↳ whenever a variable,
sub it with the value.

→ name1 = "Alice"

→ Alice = "123"

~~print(Alice)~~ → "123"

~~print(name1)~~ → "Alice"

print("Adult") ✓

~~print(Adult)~~ ✗

123

Alice

Adult

↑
variable → lookup!

Ex. → length = 3

→ width = 5

area = length * width

print(area)

Input : get input from the user

↑ display
+ "name: " + input()

Var = input("Optional text") ^{display}
want to save! ^{return a string}

- If you want a number, you need to use
 - `int(str)` → Convert str to integer
 - `float(str)` → convert str to float

Ex. width = float(input())
length = float(input())

Ex. text = "Enter age:" ^{"Enter age:"}
age = int(input(text)) → age = input(text)
print(age) → age = int(age)

Output ✓ print() ^{things you want to print} → Output

- 1) `print` ends with a new line. ("\\n")
- 2) to Print multiple things, use commas
`print("A", 123+1)` → A 123

* Option * ^{add space}

new line

L * Option *

1) end = "text" → change ending with "\n" to "text" instead new line

print("ABC") → ABC
print("xyz") → xyz

print("ABC", end=":") → ABC:xyz=
print("xyz", end="=")

print("A", end="") → AB
print("B") →

2) sep = "text" → Instead of a space, it will be "text"
print("A", "B", "C", sep="-") → A-B-C

Formatted-string (f-string)

Syntax: f" "
 f' '

* you put code/variable inside f-string and the value will be at that position.

Syntax: f" { } "
 { } { }
 ↑ ↑
 Code Code
 will be executed

Ex print(f"This is {1+1}. Hello!")
 | 1 2
 ↳ This is 2. Hello!

print("This is 2. Hello!")
 "This is 2. Hello!"

x = 5
 y = 10
 print(f"{x} + {y} = {x+y}")
 ↳ 5 + 10 = 15

* format decimal places

Syntax: f"{number:.nf}"

↑ decimal place
 - 2 decimal → .2f
 - 3 decimal → .3f

print(f"{10:.2f}") → 10.00

print(f"{10.527:.2f}") → 10.53

* format: commas 1,000,000 → f"{1000000:,.0f}"

a = f"{1+13=x}"
 print(a) → 2=x ✓

Named Constants

- It's a variable that holds a constant value
- Convention: the var name will be UPPER CASE ^{never change!!}

- Convention: the var name will be ^{never change!!} UPPER CASE

EX $PI = 3.14$

$RATE = 10$

~~$RATE = 5$~~

Minimart

↳ sell banana 5 baht
 apple 20 baht

↳ ^{input} user enters # banana and # apple

↳ output the total cost (2 decimal) ← print

↳ Must use named constant ✓

↓ ✓ BANANA_P = 5

↓ ✓ APPLE_P = 20

↓ ✓ b_n = int(input("Enter # banana: "))

↓ ✓ a_n = int(input("Enter # apple: "))

↓ ✓ total = BANANA_P * b_n + APPLE_P * a_n

↓ ✓ print(f"Total is {total:.2f}")

} 1 point (out of 100)

← - discount

Chapter 2: Decision Structures

* Structure to control when to execute code based on conditions

* Example: get a discount when total is greater than 200 baht

* Condition is either True or False

Truth value

↑
Boolean variable

Comparison Operators → result in True / False

Ex

$5 > 3$

↳ is 5 greater than 3?

↳ True

$1 > 3$

↳ is 1 greater than 3?

↳ False

* = ← assign

> greater than?

< less than?

>= greater than or equal?

<= less than or equal?

== equal?

!= not equal?

$3 == 3$

is 3 equal to 3

↓
True

ex

$4 \neq (1 + 3) \rightarrow \text{False}$

$(4 + 1) == 1 + \underline{2 * 2} \rightarrow \text{True}$

"ABC" == "ABC" → True

If structure

Syntax: if condition:
-- statement
-- statement

indicate a structure
3
indentation = spaces

True
if_block

* Only execute if_block if the condition is True.

Ex

print("A") → A		print("A") → A
if 5 > 3: → True		if 1 == 3: → False
print("B") → B		print("B")
print("C") → C		print("C")
print("Done") → Done		print("Done") → Done

space →

skip if_block

Outside

Ex. Sell donut

- donut is 30 baht.
- user enters #donut
- getadiscout if you buy 10 donuts or more of -20 baht.

D_P = 30

dn = int(input("#donut: "))

total = dn * D_P

~~total = total - 20~~
if d_n >= 10:
 [total = total - 20]
 discount
print (f" \${total:.2f}\$")